MITRC

Modern Institute of Technology & Research Centre, Alwar

Name of Faculty : Poonam Sharma

Subjects & Code : Deep Learning & 7CS4-01

Sem : VIIth Session : 2025-26 (Odd)

Branch : CSE Date of submission : 08/09/2025

Unit : III

Lecture No.	Topic Covered	Page No.	Total Pages
1	Train/Dev and Test sets	1 – 5	5
2	Bias/Variance	6-9	4
3	Overfitting and Regularization	10-15	6
4	Liner modes and Optimization	16-30	14
5	Convolution Neural Networks	31-33	3
6	RNN, Backpropagation	34-38	4
7	Pooling	39-43	5

Faculty
Academics

HOD

Dean 11/9/25

UNIT-III(Deep Learning)
Train/Dev/Test sets
Lecture No.1

Train vs. Validation vs. Test set

For training and testing purposes of our model, we should have our data broken down into three distinct dataset splits.

The Training Set

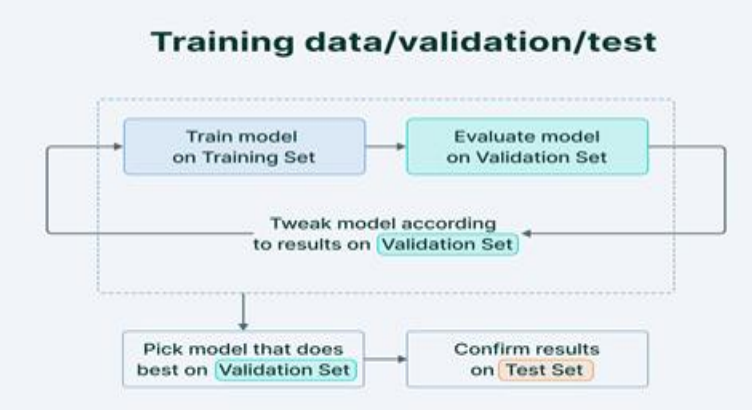
It is the set of data that is used to train and make the model learn the hidden features/patterns in the data. In each epoch, the same training data is fed to the neural network architecture repeatedly, and the model continues to learn the features of the data. The training set should have a diversified set of inputs so that the model is trained in all scenarios and can predict any unseen data sample that may appear in the future.

The Validation Set

The validation set is a set of data, separate from the training set, that is used to validate our model performance during training.This validation process gives information that helps us tune the model's hyper-parameters and configurations accordingly. It is like a critic telling us whether the training is moving in the right direction or not. The model is trained on the training set, and, simultaneously, the model evaluation is performed on the validation set after every epoch.The main idea of splitting the dataset into a validation set is to prevent our model from overfitting i.e., the model becomes really good at classifying the samples in the training set but cannot generalize and make accurate classifications on the data it has not seen before.

The Test Set

The test set is a separate set of data used to test the model *after* completing the training. It provides an unbiased final model performance metric in terms of accuracy, precision, etc. To put it simply, it answers the question of "*How well does the model perform?*"



How to split your Machine Learning data?

The creation of different samples and splits in the dataset helps us judge the true model performance.

The dataset split ratio depends on the number of samples present in the dataset and the model.

Some common inferences that can be derived on dataset split include:

- If there are several hyperparameters to tune, the machine learning model requires a larger validation set to optimize the model performance. Similarly, if the model has fewer or no hyperparameters, it would be easy to validate the model using a small set of data.
- If a model use case is such that a false prediction can drastically hamper the model performance—like falsely predicting cancer—it's better to validate the model after each epoch to make the model learn varied scenarios.
- With the increase in the dimension/features of the data, the hyperparameters of the neural network functions also increase making the model more complex. In these scenarios, a large split of data should be kept in training set with a validation set.

However, there are two major concerns while deciding on the optimum split:

- If there is less training data, the machine learning model will show high variance in training.
- With less testing data/validation data, your model evaluation/model performance statistic will have greater variance.

Essentially, you need to come up with an optimum split that suits the need of the dataset/model.



Advanced techniques for data splitting

Various data splitting techniques have been implemented in the Computer Vision literature to ensure a robust and fair way of testing machine learning models. Some of the most popular ones are explained below.

Random

Random sampling is the oldest and most popular method for dividing a dataset. As the name suggests, the dataset is shuffled, and samples are picked randomly and put in the train, validation, or the test set based on what percentage ratio is given by the user.

However, this method has a significant drawback. Random sampling works optimally on class-balanced datasets, i.e., datasets with the more or less the same number of samples in every dataset category. In the case of class-imbalanced datasets, such a data splitting method may create a bias.

For example, if a dataset has 1000 images, of which 800 belong to the category: “dog” and 200 belong to the category: “cat,” random sampling is employed to split the data into training and validation sets in an 80%-20% ratio (respectively), it might so happen that the training set consists only dog images, while the validation set consists of only cat images. Even if it’s not so extreme, there will still be an imbalance distribution among the train and validation sets.

Stratified

Stratified sampling for splitting a dataset alleviates the problem of Random Sampling in datasets with an imbalanced-class distribution. Here, the distribution of classes in each of the train, validation, and test sets is preserved.

Suppose a dataset consists of 1000 images, of which 600 are dog images and 400 are cat images. In that case, stratified sampling ensures that 60% of the images are of category “dog” and 40%

are of category "cat" in the training and validation sets. That is, if a train-validation split of 80%-20% is desired, out of the 800 images in the training set, 480 images (60%) will be of dogs, and the rest 320 (40%) will be of cats.

Let us consider another example. In object detection tasks, single images may contain several different objects belonging to disparate categories. In a dataset, some images may contain 10 examples of a dog but only 1 example of a person, while others may contain 10 people and 2 dogs, and another one might contain 5 cats and 5 dogs, with no people. In such cases, a random split of images may skew the object category-wise distribution. Stratified sampling on the other hand can partition the data in a way that the resulting object category distribution is balanced. Stratified sampling is, thus, a more fair way of data splitting, such that the machine learning model will be trained and validated on the same data distribution.

Cross-Validation

Cross-Validation or K-Fold Cross-Validation is a more robust technique for data splitting, where a model is trained and evaluated "K" times on different samples. Let us understand this with an example.Suppose we have a balanced, 2-class dataset consisting of 1000 images of raccoons and ringtails (to be used for training and validation only). Now, we want to perform a 5-Fold cross-validation. We first split the datasets into 5 equal and non-overlapping parts: each consisting of 200 images; label them as Parts 1, 2, 3, 4, and 5. Each of these subsets of 200 images consists of mutually different samples.

Now, we will create 5 complete datasets (labeled as Datasets 1-5) using Parts 1-5 in the following manner: For Dataset-1, use Part-1 as the validation set, and consolidate Parts 2-5 to create the training set; for Dataset-2, use Part-2 as the validation set, and consolidate Parts 1, 3, 4 and 5 to create the training set, and so on. Notice that since each part consists of 20% of the data of the original dataset, each of Datasets 1-5 has an 80%-20% train-validation split ratio. Generalizing, each K-Fold cross-validation dataset has (100/K)% data in its validation set (here, 100/5 = 20% was in validation set).

Using K-Fold cross-validation exposes the machine learning model (trained independently of each other) to different distributions of data ("K" times to be exact). This alleviates any bias that may occur while selecting data in the training and validation sets. Average and standard deviation values are usually reported while using K-Fold cross-validation schemes.

A simple analysis can suggest that K-Fold cross-validation also suffers from the same problem as random sampling. Data distribution may get skewed when the "K" parts or subsets of the

datasets are created. In the example above, it may so happen that Part-2 of the dataset consists of 200 images of raccoons and no images of ringtails.

Thus, the "Stratified K-Fold Cross-Validation" technique avoids such inconsistencies. Similar to stratified sampling, the class-ratio of the data is maintained while generating the "K" subsets or parts of the data. Thus, the same class distribution is carried forward when these "K" parts are coalesced to form the final complete datasets.

References

1. https://en.wikipedia.org/wiki/Training,_validation,_and_test_data_sets

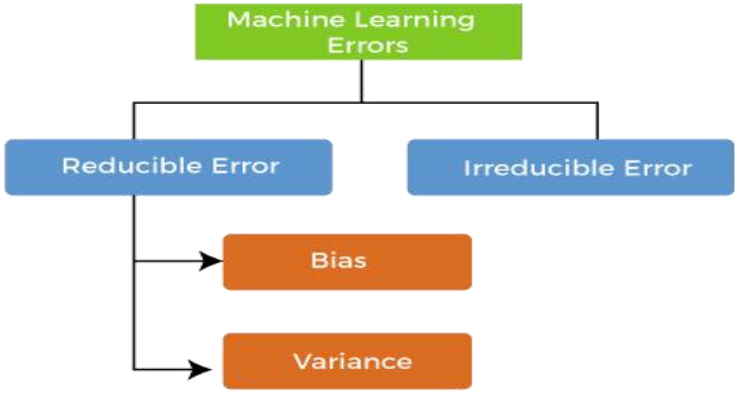
UNIT-III(Deep Learning)
Bias and Variance in Machine Learning
Lecture No.2

Machine learning is a branch of Artificial Intelligence, which allows machines to perform data analysis and make predictions. However, if the machine learning model is not accurate, it can make predictions errors, and these prediction errors are usually known as Bias and Variance. In machine learning, these errors will always be present as there is always a slight difference between the model predictions and actual predictions. The main aim of ML/data science analysts is to reduce these errors in order to get more accurate results. In this topic, we are going to discuss bias and variance, Bias-variance trade-off, Underfitting and Overfitting. But before starting, let's first understand what errors in Machine learning are?

Errors in Machine Learning

In machine learning, an error is a measure of how accurately an algorithm can make predictions for the previously unknown dataset. On the basis of these errors, the machine learning model is selected that can perform best on the particular dataset. There are mainly two types of errors in machine learning, which are:

- **Reducible errors:** These errors can be reduced to improve the model accuracy. Such errors can further be classified into bias and Variance.



- **Irreducible errors:** These errors will always be present in the model regardless of which algorithm has been used. The cause of these errors is unknown variables whose value can't be reduced.

What is Bias?

In general, a machine learning model analyses the data, find patterns in it and make predictions. While training, the model learns these patterns in the dataset and applies them to test data for prediction. ***While making predictions, a difference occurs between prediction values made by the model and actual values/expected values***, ***and this difference is known as bias errors or Errors due to bias***. It can be defined as an inability of machine learning algorithms such as Linear Regression to capture the true relationship between the data points. Each algorithm begins with some amount of bias because bias occurs from assumptions in the model, which makes the target function simple to learn. A model has either:

- **Low Bias:** A low bias model will make fewer assumptions about the form of the target function.
- **High Bias:** A model with a high bias makes more assumptions, and the model becomes unable to capture the important features of our dataset. **A high bias model also cannot perform well on new data.**

Generally, a linear algorithm has a high bias, as it makes them learn fast. The simpler the algorithm, the higher the bias it has likely to be introduced. Whereas a nonlinear algorithm often has low bias.Some examples of machine learning algorithms with low bias **are Decision Trees, k-Nearest Neighbours and Support Vector Machines**. At the same time, an algorithm with high bias is **Linear Regression, Linear Discriminant Analysis and Logistic Regression.**

Ways to reduce High Bias:

High bias mainly occurs due to a much simple model. Below are some ways to reduce the high bias:

- Increase the input features as the model is underfitted.
- Decrease the regularization term.
- Use more complex models, such as including some polynomial features.

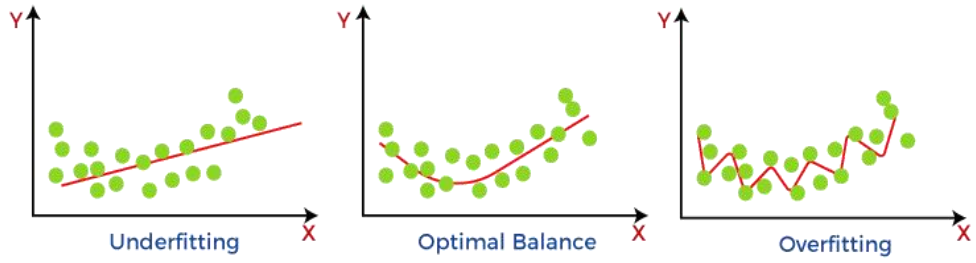
What is a Variance Error?

The variance would specify the amount of variation in the prediction if the different training data was used. In simple words, **variance tells that how much a random variable is different from its expected value.** Ideally, a model should not vary too much from one training dataset to another, which means the algorithm should be good in understanding the hidden mapping between inputs and output variables. Variance errors are either of **low variance or high variance.**

Low variance means there is a small variation in the prediction of the target function with changes in the training data set. At the same time, **High variance** shows a large variation in the prediction of the target function with changes in the training dataset.A model that shows high variance learns a lot and perform well with the training dataset, and does not generalize well with the unseen dataset. As a result, such a model gives good results with the training dataset but shows high error rates on the test dataset.Since, with high variance, the model learns too much from the dataset, it leads to overfitting of the model. A model with high variance has the below problems:

- A high variance model leads to overfitting.
- Increase model complexities.

Usually, nonlinear algorithms have a lot of flexibility to fit the model, have high variance.



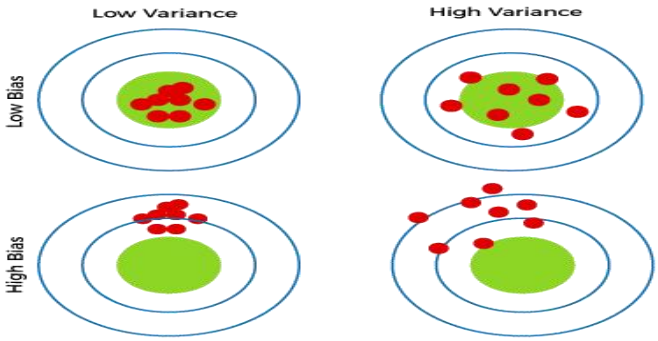
Some examples of machine learning algorithms with low variance are, **Linear Regression, Logistic Regression, and Linear discriminant analysis**. At the same time, algorithms with high variance are **decision tree, Support Vector Machine, and K-nearest neighbours.**

Ways to Reduce High Variance:

- Reduce the input features or number of parameters as a model is overfitted.
- Do not use a much complex model.
- Increase the training data.
- Increase the Regularization term.

Different Combinations of Bias-Variance

There are four possible combinations of bias and variances, which are represented by the below diagram:



1. **Low-Bias, Low-Variance:** The combination of low bias and low variance shows an ideal machine learning model. However, it is not possible practically.
2. **Low-Bias, High-Variance:** With low bias and high variance, model predictions are inconsistent and accurate on average. This case occurs when the model learns with a large number of parameters and hence leads to an **overfitting**
3. **High-Bias, Low-Variance:** With High bias and low variance, predictions are consistent but inaccurate on average. This case occurs when a model does not learn well with the training dataset or uses few numbers of the parameter. It leads to **underfitting** problems in the model.
4. **High-Bias, High-Variance:** With high bias and high variance, predictions are inconsistent and also inaccurate on average.

References

1. https://www.geeksforgeeks.org/bias-vs-variance-in-machine-learning/

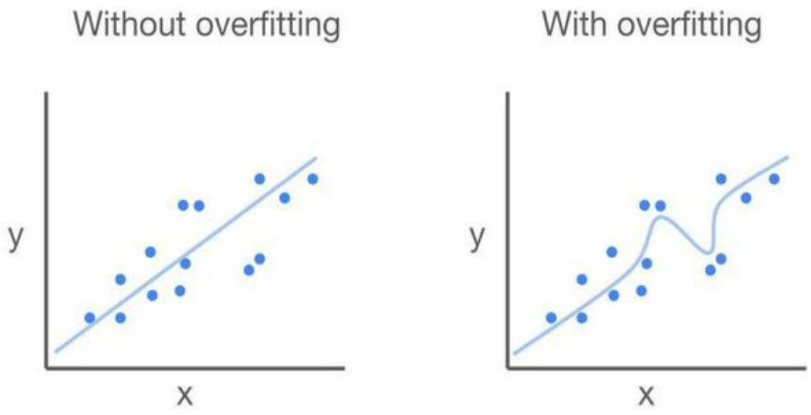
UNIT-III(Deep Learning)
Overfitting and Regularization
Lecture No.3

The effectiveness of a machine learning model is measured by its ability to make accurate predictions and minimize prediction errors. An ideal or good machine learning model should be able to perform well with new input data, allowing us to make accurate predictions about future data that the model has not seen before. This ability to work well with future data (unseen data) is known as **generalization**. To consider how well a machine learning model learns and generalizes to new data, we are going to examine the concept of **overfitting** which is the key factor that can significantly impact the performance of machine learning algorithms on future data, and also going to discuss the Regularization concept which will try to avoid overfitting in machine learning.

Overfitting in Machine Learning

In Machine learning, there is a term called **train data** and **test data** which machine learning model will learn from train data and try to predict the test data based on its learning. Overfitting is a concept in machine learning which states a common problem that occurs when a model **learns the train data too well** including the noisy data, resulting in **poor generalization performance on test data**. Overfit models don't generalize, which is the ability to apply knowledge to different situations.

Suppose we are training a linear regression model to predict the price of a house based on its square feet and with few specifications. We collect a dataset of houses with their square feet and sale price. We then train our linear regression model on this dataset. Generally in linear regression algorithms, it draws a straight that best fits the data points by minimizing the difference between predicted and actual values. The goal is to make a straight line that captures the main pattern in the dataset . This way, it can predict new points more accurately. But sometimes we come across overfitting in linear regression as bending that straight line to fit exactly with a few points on the pattern which is shown below fig.1. This might look perfect for those points while training but doesn't work well for other parts of the pattern when come to model testing.



Reason for Overfitting

Let's discuss what are the reasons that cause overfitting to the machine learning model which are listed below,

1. Assigning a complex model that has too many parameters is more likely to overfit the training data.
2. when the training dataset is too small the model may not be able to learn the underlying patterns in the data and may start to learn the noise in the data as well.
3. when the training data is highly imbalanced with one output class so model may learn to bias its predictions toward the majority class.
4. In feature engineering, the features are not properly scaled or engineered thus the model leads to overfitting.
5. In feature selection, selected features for training the model are not relevant to the target variable, it is more likely to overfit the training data.
6. If the model trains the data too long, it may start to learn the noise in the data and it tends to overfitting.

Techniques to avoid overfitting

Let's discuss some of the techniques to avoid overfitting to a machine learning model that is listed below,

1. Do not make the model too complex, make it simple by removing unnecessary features or reducing the number of parameters.
2. Increasing the size of training data by adding more information can able to avoid overfitting.
3. Make train data clean by removing the noise and outliers this can able to avoid overfitting.
4. Try to make separate validation datasets to evaluate the model performance if the model performs poorly on the validation set, it is likely overfitting the training data.

5. Use Ensemble methods, that combine multiple models to provide accurate prediction which can help to reduce the risk of overfitting.
6. Use cross-validation, a technique which splits train data repeatedly into many folds and the remaining fold as validation data this can help to reduce overfitting by ensuring that the model is evaluated on a variety of different data splits.
7. By applying regularization techniques, which add a penalty to the model's complexity, this can help to prevent overfitting.

Regularization Technique

Regularization is a technique in machine learning that helps prevent from overfitting. It works by introducing penalties term or constraints on the model's parameters during training. These penalties term encourage the model to avoid extreme or overly complex parameter values. By doing so, regularization prevents the model from fitting the training data too closely, which is a common cause of overfitting. Instead, it promotes a balance between model complexity and performance, leading to better generalization on new, unseen data.

How Regularization used to prevent overfitting?

1. By introducing the regularization term in loss function that act like a constrain function of the model's parameter.This function penalize certain parameter values in model, discouraging them from becoming too large or complex.
2. Regularization introduces a trade-off between fitting the training data and keeping the model's parameters small. The strength of regularization is controlled by a hyperparameter, often denoted as lambda (λ). A higher λ value leads to stronger regularization and a simpler model.
3. Regularization techniques help control the complexity of the model. They make the model more robust by constraining the parameter space. This results in smoother decision boundaries in the case of classification and smoother functions in regression, reducing the potential for overfitting.
4. Regularization oppose overfitting by discouraging the model from fitting the training data too closely. It prevents parameters from taking extreme values, which might be necessary to fit the training data.

Let's discuss about two common techniques that involve in regularization which can prevent model from overfitting:

- **L1 Regularization**

- **L2 Regularization**

L1 Regularization

L1 regularization, also known as Lasso **(Least Absolute Shrinkage and Selection Operator)** regularization, is a statistical technique used in machine learning to avoid overfitting. It is used to add a penalty term to the model's loss function. This penalty term encourages the model to keep some of its coefficients exactly equal to zero, effectively performing feature selection. L1 regularization is employed to prevent overfitting, simplify the model, and enhance its generalization to new, unseen data. It is particularly useful when dealing with datasets containing many features, as it helps identify and focus on the most essential ones, disregarding less influential variables.

Let's derive the mathematical formulation for L1 regularization (Lasso) in simple terms.

In linear regression, the standard model's goal is to minimize the mean squared error (MSE), represented as:

$$MSE = \Sigma(y - \hat{y})^2$$

In the above equation, 'y' is the actual target, and 'ŷ' is the predicted target. Now, to add L1 regularization, we introduce a new term to the model's loss function:

$$Loss = MSE + \alpha * \Sigma|w|$$

Here, 'w' represents the model's coefficients, and 'α' is the regularization strength, a hyperparameter that controls how much regularization is applied. The term **Σ|w|** sums up the **absolute values of all the coefficients**.What this additional term does is encourage the model to have some of its **coefficients exactly equal to zero**. It's like a feature selector it helps the model pick only the most important features, effectively ignoring the less relevant ones.

The larger the 'α' value, the stronger the regularization, and the more coefficients will become zero. This balance between the **MSE (model fitting the data)** and the **Σ|w| term (coefficient sparsity)** ensures that the model remains relatively simple and less prone to overfitting while still fitting the data effectively.

L2 Regularization

L2 regularization, often referred to as Ridge regularization, is a is a statistical technique used in machine learning to avoid overfitting. It involves adding a penalty term to the model's loss function, encouraging the model's coefficients to be small but not exactly

zero. Unlike L1 regularization, which can lead to some coefficients becoming precisely zero, L2 regularization aims to keep all coefficients relatively small. This technique helps prevent overfitting, improves model generalization, and maintains a balance between bias and variance. L2 regularization is especially beneficial when dealing with datasets with numerous features, as it helps control the influence of each feature, contributing to more robust and stable model performance.

Let's derive the mathematical formulation for L2 regularization (Ridge) in simple terms.

In linear regression, the standard model's goal is to minimize the mean squared error (MSE), which is represented as:

$$MSE = \Sigma(y - \hat{y})^2$$

Here, 'y' is the actual target, and 'ŷ' is the predicted target.Now, to add L2 regularization, we introduce a new term to the model's loss function:

$$Loss = MSE + \alpha * \Sigma(w^2)$$

In this equation, 'w' represents the model's coefficients, and 'α' is the regularization strength, a hyperparameter that controls how much regularization is applied. The term **Σ(w^2)** adds up the **squares of all the coefficients**. What this extra term does is encourage the model to keep its **coefficient values small**. It helps to prevent any one feature from having an overly dominant effect on the predictions. In other words, it adds a cost to having large coefficients.

The larger the 'α' value, the stronger the regularization, and the smaller the coefficients will become. This balance between the **MSE (model fitting the data)** and the **Σ(w^2) term (model's coefficients)** ensures that the model doesn't become too complex, preventing overfitting while still fitting the data effectively.

How L1 and L2 Regularization used to prevent overfitting

1. L1 regularization helps in automatically selecting the most important features by setting the corresponding coefficients to zero this help to reduce the model complexity and thus prevent from overfitting.
2. L1 regularization corresponds to a diamond-shaped constraint in parameter space. This constraint leads to parameter values being pushed towards the coordinate axes, resulting in a simpler model which avoid from overfitting.
3. L2 regularization corresponds to a circular constraint in parameter space. This constraint leads to parameter values being pushed towards the origin, which results in a more stable and well-conditioned model that avoid from overrfitting.

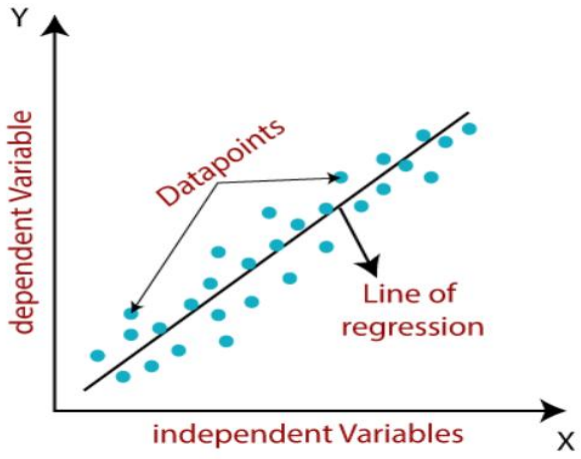
4. L2 regularization, is often used to handle multicollinearity and helps to distribute the importance of correlated features more evenly this avoid complexity of model which prevent from overfitting.

References

1. https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/

UNIT-III(Deep Learning)
Linear models and optimization
Lecture No.4

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc. Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable. The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Mathematically, we can represent a linear regression as:

$$y= a_0+a_1x+ \varepsilon$$

Here,

Y= Dependent Variable (Target Variable),X= Independent Variable (predictor Variable)
a0= intercept of the line (Gives an additional degree of freedom)
a1=Linear regression coefficient (scale factor to each input value).
ε = random error.
The values for x and y variables are training datasets for Linear Regression model representation.

Types of Linear Regression

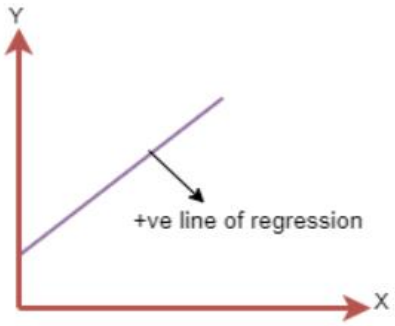
Linear regression can be further divided into two types of the algorithm:

- **Simple Linear Regression:**If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
- **Multiple Linear regression:**If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Linear Regression Line

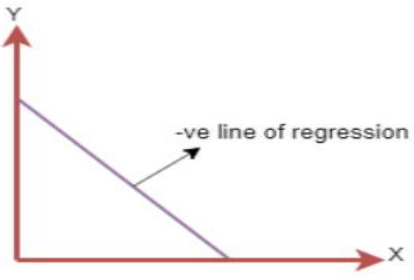
A linear line showing the relationship between the dependent and independent variables is called a **regression line**. A regression line can show two types of relationship:

- **Positive Linear Relationship:** If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.



The line equation will be: $Y= a_0+a_1x$

- **Negative Linear Relationship:** If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.



The line of equation will be: $Y= -a_0+a_1x$

Finding the best fit line:

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0, a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0, a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

For the above linear equation, MSE can be calculated as:

$$MSE = 1\frac{1}{N}\sum_{i=1}^{n}(y_i - (a_1 x_i + a_0))^2$$

Where,

N=Total number of observation

Yi = Actual value

$(a1x_i+a_0)$= Predicted value.

Residuals: The distance between the actual value and predicted values is called residual. If the observed points are far from the regression line, then the residual will be high, and so cost function will high. If the scatter points are close to the regression line, then the residual will be small and hence the cost function.

Gradient Descent:

- Gradient descent is used to minimize the MSE by calculating the gradient of the cost function.
- A regression model uses gradient descent to update the coefficients of the line by reducing the cost function.

- It is done by a random selection of values of coefficient and then iteratively update the values to reach the minimum cost function.

Model Performance:

The Goodness of fit determines how the line of regression fits the set of observations. The process of finding the best model out of various models is called **optimization**. It can be achieved by below method:

1. R-squared method:

- R-squared is a statistical method that determines the goodness of fit.
- The high value of R-square determines the less difference between. It measures the strength of the relationship between the dependent and independent variables on a scale of 0-100%.
- Predicted values and actual values and hence represents a good model.
- It is also called a **coefficient of determination,** or **coefficient of multiple determination** for multiple regression.
- It can be calculated from the below formula:

$$\text{R-squared} = \frac{\text{Explained variation}}{\text{Total Variation}}$$

Assumptions of Linear Regression

Below are some important assumptions of Linear Regression. These are some formal checks while building a Linear Regression model, which ensures to get the best possible result from the given dataset.

- **Linear relationship between the features and target:** Linear regression assumes the linear relationship between the dependent and independent variables.
- **Small or no multicollinearity between the features:** Multicollinearity means high-correlation between the independent variables. Due to multicollinearity, it may difficult to find the true relationship between the predictors and target variables. Or we can say, it is difficult to determine which predictor variable is affecting the target variable and which is not. So, the model assumes either little or no multicollinearity between the features or independent variables.
- **Homoscedasticity Assumption:** Homoscedasticity is a situation when the error term is the same for all the values of independent variables. With homoscedasticity, there should be no clear pattern distribution of data in the scatter plot.
- **Normal distribution of error terms:** Linear regression assumes that the error term should follow the normal distribution pattern. If error terms are not normally

distributed, then confidence intervals will become either too wide or too narrow, which may cause difficulties in finding coefficients.

It can be checked using the **q-q plot**. If the plot shows a straight line without any deviation, which means the error is normally distributed.

- **No autocorrelations:** The linear regression model assumes no autocorrelation in error terms. If there will be any correlation in the error term, then it will drastically reduce the accuracy of the model. Autocorrelation usually occurs if there is a dependency between residual errors.

References

1. https://www.geeksforgeeks.org/ml-linear-regression/

UNIT-III(Deep Learning)
Vanishing/exploding gradients
Lecture No.5

What is Vanishing Gradient?

The vanishing gradient problem is a challenge that emerges during backpropagation when the derivatives or slopes of the activation functions become progressively smaller as we move backward through the layers of a neural network. This phenomenon is particularly prominent in deep networks with many layers, hindering the effective training of the model. The weight updates becomes extremely tiny, or even exponentially small, it can significantly prolong the training time, and in the worst-case scenario, it can halt the training process altogether.

Why the Problem Occurs?

During backpropagation, the gradients propagate back through the layers of the network, they decrease significantly. This means that as they leave the output layer and return to the input layer, the gradients become progressively smaller. As a result, the weights associated with the initial levels, which accommodate these small gradients, are updated little or not at each iteration of the optimization process.

The vanishing gradient problem is particularly associated with the sigmoid and hyperbolic tangent (tanh) activation functions because their derivatives fall within the range of 0 to 0.25 and 0 to 1, respectively. Consequently, extreme weights becomes very small, causing the updated weights to closely resemble the original ones. This persistence of small updates contributes to the vanishing gradient issue.

The sigmoid and tanh functions limit the input values to the ranges [0,1] and [-1,1], so that they saturate at 0 or 1 for sigmoid and -1 or 1 for Tanh. The derivatives at points becomes zero as they are moving. In these regions, especially when inputs are very small or large, the gradients are very close to zero. While this may not be a major concern in shallow networks with a few layers, it is a more pronounced issue in deep networks. When the inputs fall in saturated regions, the gradients approach zero, resulting in little update to the weights of the previous layer. In simple networks this does not pose much of a problem, but as more layers are added, these small gradients, which multiply

between layers, decay significantly and consequently the first layer tears very slowly , and hinders overall model performance and can lead to convergence failure.

How can we identify?

Identifying the vanishing gradient problem typically involves monitoring the training dynamics of a deep neural network.

- One key indicator is observing model weights **converging to 0** or stagnation in the improvement of the model's performance metrics over training epochs.
- During training, if the **loss function fails to decrease** significantly, or if there is erratic behavior in the learning curves, it suggests that the gradients may be vanishing.
- Additionally, examining the gradients themselves during backpropagation can provide insights. **Visualization techniques**, such as gradient histograms or norms, can aid in assessing the distribution of gradients throughout the network.

How can we solve the issue?

- **Batch Normalization** : Batch normalization normalizes the inputs of each layer, reducing internal covariate shift. This can help stabilize and accelerate the training process, allowing for more consistent gradient flow.
- **Activation function**: Activation function like **Rectified Linear Unit (ReLU)** can be used. With **ReLU,** the gradient is 0 for negative and zero input, and it is 1 for positive input, which helps alleviate the vanishing gradient issue. Therefore, ReLU operates by replacing poor enter values with 0, and 1 for fine enter values, it preserves the input unchanged.
- **Skip Connections and Residual Networks (ResNets)**: Skip connections, as seen in ResNets, allow the gradient to bypass certain layers during backpropagation. This facilitates the flow of information through the network, preventing gradients from vanishing.
- **Long Short-Term Memory Networks (LSTMs) and Gated Recurrent Units (GRUs)**: In the context of recurrent neural networks (RNNs), architectures like LSTMs and GRUs are designed to address the vanishing gradient problem in sequences by incorporating gating mechanisms .
- **Gradient Clipping**: Gradient clipping involves imposing a threshold on the gradients during backpropagation. Limit the magnitude of gradients during backpropagation, this can prevent them from becoming too small or exploding, which can also hinder learning.

What is Exploding Gradient?

The exploding gradient problem is a challenge encountered during training deep neural networks. It occurs when the gradients of the network's loss function with respect to the weights (parameters) become excessively large.

Why Exploding Gradient Occurs?

The issue of exploding gradients arises when, during backpropagation, the derivatives or slopes of the neural network's layers grow progressively larger as we move backward. This is essentially the opposite of the vanishing gradient problem.

The root cause of this problem lies in the weights of the network, rather than the choice of activation function. High weight values lead to correspondingly high derivatives, causing significant deviations in new weight values from the previous ones. As a result, the gradient fails to converge and can lead to the network oscillating around local minima, making it challenging to reach the global minimum point.

In summary, exploding gradients occur when weight values lead to excessively large derivatives, making convergence difficult and potentially preventing the neural network from effectively learning and optimizing its parameters.As we discussed earlier, the update for the weights during backpropagation in a neural network is given by:

$$W_i = LW_i$$

Where, L is the learning rate.

The exploding gradient problem occurs when the gradients become very large during backpropagation. This is often the result of gradients greater than 1, leading to a rapid increase in values as you propagate them backward through the layers.Mathematically, the update rule becomes problematic when $W_i > 1,$, causing the weights to increase exponentially during training.

How can we identify the problem?

Identifying the presence of exploding gradients in deep neural network requires careful observation and analysis during training. Here are some key indicators:

- The loss function exhibits erratic behavior, oscillating wildly instead of steadily decreasing suggesting that the network weights are being updated excessively by large gradients, preventing smooth convergence.

- The training process encounters “NaN” (Not a Number) values in the loss function or other intermediate calculations.
- If network weights, during training exhibit significant and rapid increases in their values, it suggests the presence of exploding gradients.
- Tools like TensorBoard can be used to visualize the gradients flowing through the network.

How can we solve the issue?

- **Gradient Clipping**: It sets a maximum threshold for the magnitude of gradients during backpropagation. Any gradient exceeding the threshold is clipped to the threshold value, preventing it from growing unbounded.
- **Batch Normalization:** This technique normalizes the activations within each mini-batch, effectively scaling the gradients and reducing their variance. This helps prevent both vanishing and exploding gradients, improving stability and efficiency.

References

1. https://neptune.ai/blog/vanishing-and-exploding-gradients-debugging-monitoring-fixing

UNIT-III(Deep Learning)

Logistic Regression

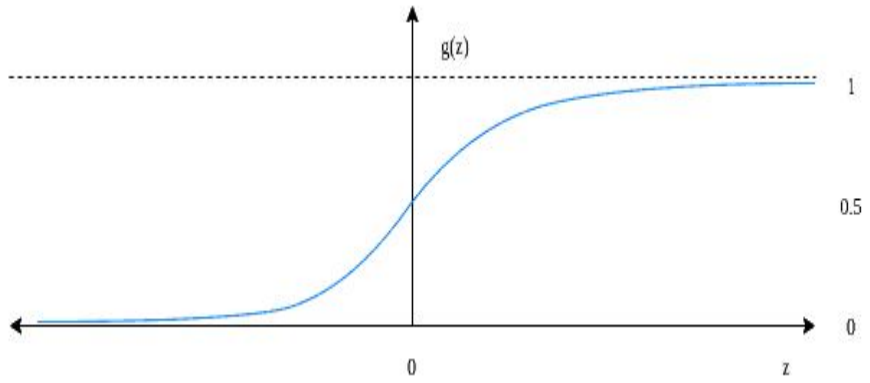
Lecture No.6

We use logistic regression to solve classification problems where the outcome is a discrete variable. Usually, we use it to solve binary classification problems. As the name suggests, binary classification problems have two possible outputs.We utilize the sigmoid function (or logistic function) to map input values from a wide range into a limited interval. Mathematically, the sigmoid function is:

$$y = g(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$$

This formula represents the probability of observing the output Y of a Bernoulli random variable. This variable is either 0 or 1 y ε (0,1)

It squeezes any real number to the (0,1) open interval. Thus, it's better suited for classification. Moreover, it's less sensitive to outliers, unlike linear regression:



Applying the sigmoid function on 0 gives 0.5. The output becomes as the input approaches 'inf'. Conversely, sigmoid becomes 0 as the input approaches '–inf'.More formally, we define the logistic regression model for binary classification problems. We choose the hypothesis function to be the sigmoid function:

$$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$$

Here, θ denotes the parameter vector. For a model containing n features, we have $\theta = [\theta_0, \theta_1, ..., \theta_n]$ containing $n+1$ parameters. The hypothesis function approximates the estimated probability of the actual output being equal to 1. In other words:

$$P(y=1|\theta,x)=g(z)=\frac{1}{1+e^{-\theta^T x}}$$

and

$$P(y=0|\theta,x)=1-g(z)=1-\frac{1}{1+e^{-\theta^T x}}=\frac{1}{1+e^{\theta^T x}}$$

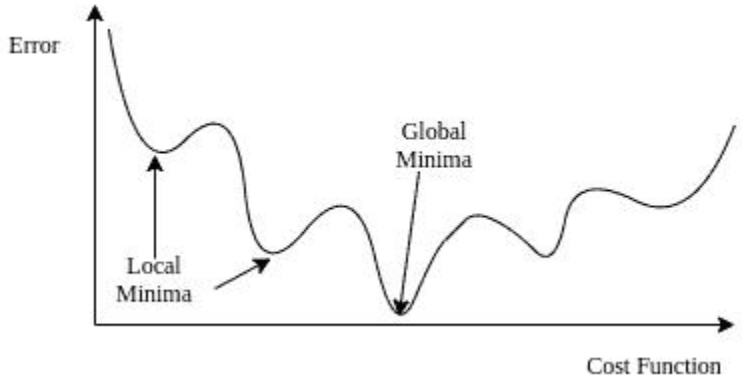
More compactly, this is equivalent to:

$$P(y|\theta,x)=\left(\frac{1}{1+e^{-\theta^T x}}\right)^{y}\times\left(1-\left(\frac{1}{1+e^{\theta^T x}}\right)\right)^{1-y}$$

Cost Function

The cost function summarizes how well the model is behaving. In other words, we use the cost function to measure how close the model's predictions are to the actual outputs.

In linear regression, we use mean squared error (MSE) as the cost function. But in logistic regression, using the mean of the squared differences between actual and predicted outcomes as the cost function might give a wavy, non-convex solution; containing many local optima:

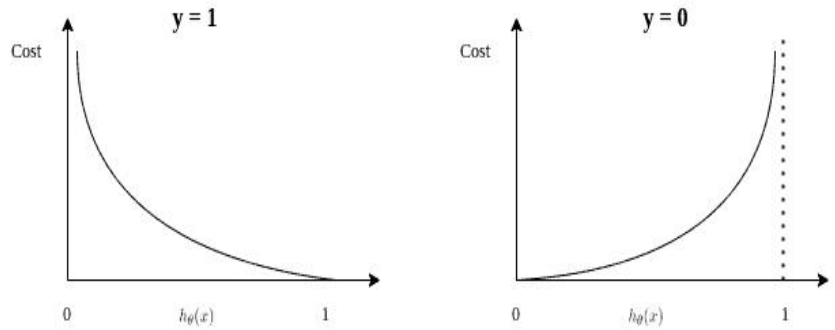


In this case, finding an optimal solution with the gradient descent method is not possible. **Instead, we use a logarithmic function to represent the cost of logistic regression.** It is guaranteed to be convex for all input values, containing only one minimum, allowing us to run the gradient descent algorithm.

When dealing with a binary classification problem, the logarithmic cost of error depends on the value of y. We can define the cost for two cases separately:

$$cost(h_\theta(x),y)=\begin{cases}-log(h_\theta(x)) & \text{, if } y=1\\ -log(1-h_\theta(x)) & \text{, if } y=0\end{cases}$$

Which then results in:



Because when the actual outcome $y = 1$, the cost is 0 for $h_\theta(x) = 1$ and takes the maximum value for $h_\theta(x) = 0$. Similarly, if $y = 0$, the cost is 0 for $h_\theta(x) = 0$.

As the output can either be 0 or 1, we can simplify the equation to be:

$$cost(h_\theta(x), y) = -y^{(i)} \times log(h_\theta(x^{(i)})) - (1 - y^{(i)}) \times log(h_\theta(x^{(i)}))$$

For m observations, **we can calculate the cost as:**

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)} \times log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \times log(h_\theta(x^{(i)}))\right]$$

Minimizing the Cost with Gradient Descent

Gradient descent is an iterative optimization algorithm, which finds the minimum of a differentiable function. In this process, we try different values and update them to reach the optimal ones, minimizing the output.

In this article, we can apply this method to the cost function of logistic regression. This way, we can find an optimal solution minimizing the cost over model parameters:

$$\min_\theta J(\theta)$$

As already explained, we're using the sigmoid function as the hypothesis function in logistic regression.

As already explained, we're using the sigmoid function as the hypothesis function in logistic regression.

Assume we have a total of n features. In this case, we have n parameters for the θ vector. To minimize our cost function, we need to run the gradient descent on each parameter θ_j:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Furthermore, we need to update each parameter simultaneously for each iteration. In other words, we need to loop through the parameters $\theta_0, \theta_1, .., \theta_n$ in vector $\theta = [\theta_0, \theta_1, ..., \theta_n]$.

To complete the algorithm, we need the value of $\frac{\partial}{\partial \theta_j} J(\theta)$, which is:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Plugging this into the gradient descent function leads to the update rule:

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Surprisingly, the update rule is the same as the one derived by using the sum of the squared errors in linear regression. As a result, we can use the same gradient descent formula for logistic regression as well.

By iterating over the training samples until convergence, we reach the optimal parameters θ leading to minimum cost.

References

1. https://www.simplilearn.com/tutorials/machine-learning-tutorial/logistic-regression-in-python

UNIT-III(Deep Learning)
CNN and RNN
Lecture No.7

What's the difference between CNN and RNN?

The main difference between a CNN and an RNN is the ability to process temporal information — data that comes in sequences, such as a sentence. Recurrent neural networks are designed for this very purpose, while convolutional neural networks are incapable of effectively interpreting temporal information. As a result, CNNs and RNNs are used for completely distinct purposes, and there are differences in the structures of the neural networks themselves to fit those different use cases.

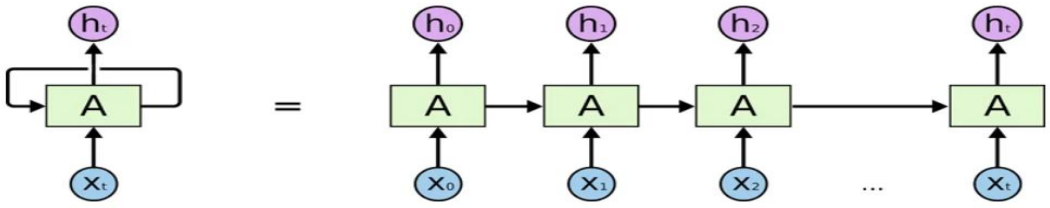
CNNs employ filters within convolutional layers to transform data (more on that later), whereas RNNs are predictive, reusing activation functions from other data points in the sequence to generate the next output in a series.

What is a recurrent neural network?

Recurrent neural networks are designed to interpret temporal or sequential information. These networks use other data points in a sequence to make better predictions. They do this by taking in input and reusing the activations of previous nodes or later nodes in the sequence to influence the output.

Entity extraction in text is a great example of how data in different parts of a sequence can affect each other. With entities, the words that come before and after the entity in the sentence have a direct effect on how they are classified. In order to deal with temporal or sequential data, like sentences, we have to use algorithms that are designed to learn from past data and 'future data' in the sequence.

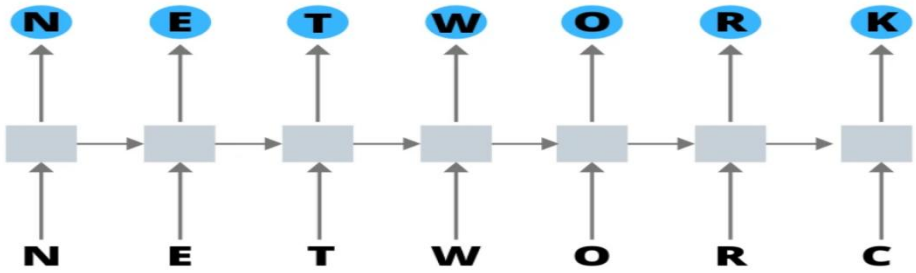
Take, for example, the following text: *President **Roosevelt** was one of the most influential presidents in American history. However, **Roosevelt** Street in Manhattan was not named after him.*



In the first sentence, Roosevelt should be labeled as a *person entity*, but in the second sentence it should be labeled as a street name or a location. Knowing these distinctions is not possible without taking into account the words before them ("President"), and after them ("Street"). *Image of RNN structure.*

RNNs for autocorrect

To dive a little deeper into how an RNN works, you need look no further than autocorrect systems. At a basic level, autocorrect systems take the word you've typed as input. Using that input, the system makes a prediction as to whether the spelling is correct or incorrect. If the word doesn't match any words in the database, or doesn't fit in the context of the sentence, the system then predicts what the correct word might be. Let's visualize how this process works making use of an RNN.



The RNN would take in two sources of input. The first input is the letter you've typed. The second input would be the activation functions corresponding to the previous letters you typed. Let's say you wanted to type "network," but typed "networc" by mistake. The system takes in the activation functions of the previous letters "networ," and the current letter you've inputted, "c." It then offers "k" as the correct output for the last letter.

This is just one simplified example of how RNNs can work for spelling correction. Today, data scientists use RNNs to do an array of incredible things — from generating text and captions for images to creating music and predicting stock market fluctuations, RNNs have endless potential use cases.

What is a convolutional neural network?

Convolutional neural networks are one of the most common types of neural networks used in computer vision to recognize objects and patterns in images. One of their defining traits is the use of filters within convolutional layers.

Convolutional layers

CNNs have unique layers called convolutional layers that separate them from RNNs and other neural networks. Within a convolutional layer, the input is transformed before being passed to the next layer. A CNN transforms the data by using filters.

What are filters in convolutional neural networks?

A filter is simply a matrix of randomized number values like you see in the diagram below.

0.230	0.380	0.971
0.402	0.119	0.886
0.693	0.563	0.771

Image of an example 3 x 3 filter.

The number of rows and columns in the filter can vary and is dependent on the use case and data being processed. Within a convolutional layer, there are a number of filters that move through an image. This process is referred to as convolving, which is a term with origins in mathematics involving the act of combining. The filter convolves the pixels of the image, changing their values before passing the data on to the next layer in the network.

How do filters work?

To understand how filters transform data, let's take a look at how we can train a CNN to recognize handwritten digits. Below is an enlarged version of a 28 x 28 pixel image of the number seven from the MNIST dataset, which includes images of handwritten digits.

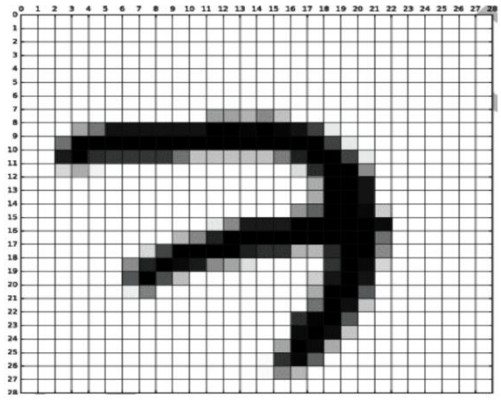


Image of a handwritten "7" from the MNIST dataset.

Below is the same image converted into its pixel values — a measurement used to describe relative contrast between pixels.



Image of a handwritten "7" from the MNIST dataset converted into its pixel values.

As the filter convolves through the image, the matrix of values in the filter line up with the pixel values of the image and the dot product of those values is taken.

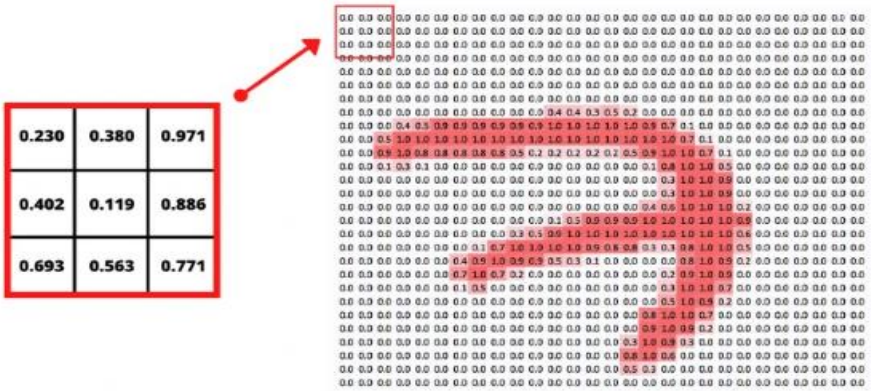


Image of dot product calculation for handwritten "7" from the MNIST dataset.

The filter moves, or convolves, through each 3 x 3 matrix of pixels until all the pixels have been covered. The dot product of each calculation is then used as input for the next layer. Initially, the values in the filter are randomized. As a result, the first passes or convolutions act as a training phase and the initial output isn't very useful. After each iteration, the CNN adjusts these values automatically using a loss function. As the training progresses, the CNN continuously adjusts the filters. By doing so, it is able to distinguish edges, curves, textures and more patterns and features of the image.

While this is an impressive feat, in order to implement loss functions, a CNN needs to be given examples of correct output in the form of labeled training data. Typically CNNs benefit from transfer learning, which is a practice that involves amassing knowledge about a problem and applying it to similar cases in the future. When transfer learning can't be applied, many convolutional neural networks require exorbitant amounts of labeled data.

References

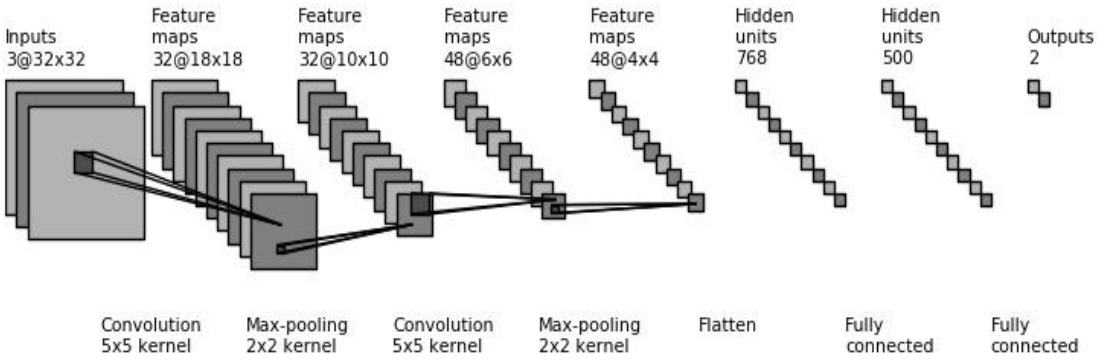
1. https://en.wikipedia.org/wiki/Recurrent_neural_network

UNIT-III(Deep Learning)

Backpropagation In Convolutional Neural Networks

Lecture No. 8 & 9

Convolutional neural networks (CNNs) are a biologically-inspired variation of the multilayer perceptrons (MLPs). Neurons in CNNs share weights unlike in MLPs where each neuron has a separate weight vector. This sharing of weights ends up reducing the overall number of trainable weights hence introducing sparsity.



Utilizing the weights sharing strategy, neurons are able to perform convolutions on the data with the convolution filter being formed by the weights. This is then followed by a pooling operation which as a form of non-linear down-sampling, progressively reduces the spatial size of the representation thus reducing the amount of computation and parameters in the network.

Existing between the convolution and the pooling layer is an activation function such as the ReLu layer; a non-saturating activation is applied element-wise, i.e. $f(x) = max(0, x)$ thresholding at zero. After several convolutional and pooling layers, the image size (feature map size) is reduced and more complex features are extracted. Eventually with a small enough feature map, the contents are squashed into a one dimension vector and fed into a fully-connected MLP for processing. The last layer of this fully-connected MLP seen as the output, is a loss layer which is used to specify how the network training penalizes the deviation between the predicted and true labels.

Cross-correlation

Given an input image I and a filter (kernel) K of dimensions $k_1 \times k_2$, the cross-correlation operation is given by:

$$(I \otimes K)_{ij} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i+m, j+n) K(m,n) \quad (1)$$

Convolution

Given an input image I and a filter (kernel) K of dimensions $k_1 \times k_2$, the cross-correlation operation is given by:

$$(I * K)_{ij} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i-m, j-n) K(m,n) \quad (2)$$

$$= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i+m, j+n) K(-m,-n) \quad (3)$$

From Eq. 3 it is easy to see that convolution is the same as cross-correlation with a flipped kernel i.e: for a kernel K where $K(-m,-n) == K(m,n)$.

Convolution Neural Networks - CNNs

CNNs consists of convolutional layers which are characterized by an input map I, a bank of filters K and biases b.

In the case of images, we could have as input an image with height H, width W and $C = 3$ channels (red, blue and green) such that $I \in \mathbb{R}^{H \times W \times C}$. Subsequently for a bank of D filters we have $K \in \mathbb{R}^{k_1 \times k_2 \times C \times D}$ and biases $b \in \mathbb{R}^D$, one for each filter.

The output from this convolution procedure is as follows:

$$(I * K)_{ij} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \sum_{c=1}^{C} K_{m,n,c} \cdot I_{i+m,j+n,c} + b \quad (4)$$

The convolution operation carried out here is the same as cross-correlation, except that the kernel is “flipped” (horizontally and vertically).

For the purposes of simplicity we shall use the case where the input image is grayscale i.e single channel ,C=1. The Eq. 4 will be transformed to:

$$(I * K)_{ij} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} K_{m,n} \cdot I_{i+m,j+n} + b \quad (5)$$

Notation

To help us explore the forward and backpropagation, we shall make use of the following notation:

1. l is the lth layer where $l = 1$ is the first layer and $l = L$ is the last layer.
2. Input x is of dimension $H \times W$ and has i by j as the iterators
3. Filter or kernel w is of dimension $k_1 \times k_2$ has m by n as the iterators
4. $w^l_{m,n}$ is the weight matrix connecting neurons of layer l with neurons of layer $l-1$.
5. b^l is the bias unit at layer l.
6. $x^l_{i,j}$ is the convolved input vector at layer l plus the bias represented as

$$x^l_{i,j} = \sum_m \sum_n w^l_{m,n} o^{l-1}_{i+m,j+n} + b^l$$

7. $o^l_{i,j}$ is the output vector at layer l given by

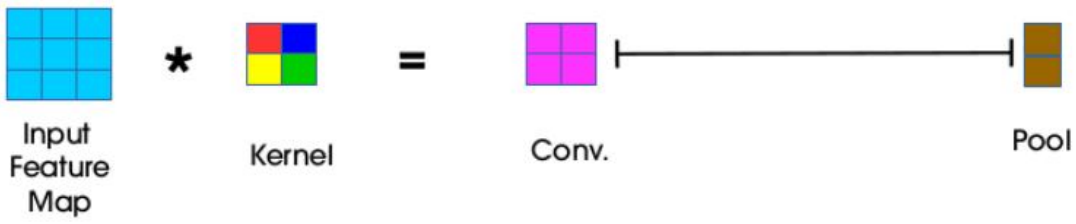
$$o^l_{i,j} = f(x^l_{i,j})$$

8. $f(\cdot)$ is the activation function. Application of the activation layer to the convolved input vector at layer l is given by $f(x^l_{i,j})$

Forward Propagation

To perform a convolution operation, the kernel is flipped 180◦ and slid across the input feature map in equal and finite strides. At each location, the product between each element of the kernel and the input input feature map element it overlaps is computed and the results summed up to obtain the output at that current location.

This procedure is repeated using different kernels to form as many output feature maps as desired. The concept of weight sharing is used as demonstrated in the diagram below:



Units in convolutional layer illustrated above have receptive fields of size 4 in the input feature map and are thus only connected to 4 adjacent neurons in the input layer. This is the idea of **sparse connectivity** in CNNs where there exists local connectivity pattern between neurons in adjacent layers.

The color codes of the weights joining the input layer to the convolutional layer show how the kernel weights are distributed (shared) amongst neurons in the adjacent layers. Weights of the same color are constrained to be identical. The convolution process here is usually expressed as a cross-correlation but with a flipped kernel. In the diagram below we illustrate a kernel that has been flipped both horizontally and vertically:



The convolution equation of the input at layer 'l' is given by:

$$x_{i,j}^{l} = \text{rot}_{180^\circ}\left\{w_{m,n}^{l}\right\} * o_{i,j}^{l-1} + b_{i,j}^{l} \quad (6)$$

$$x_{i,j}^{l} = \sum_{m}\sum_{n} w_{m,n}^{l} o_{i+m,j+n}^{l-1} + b_{i,j}^{l} \quad (7)$$

$$o_{i,j}^{l} = f(x_{i,j}^{l}) \quad (8)$$

Error

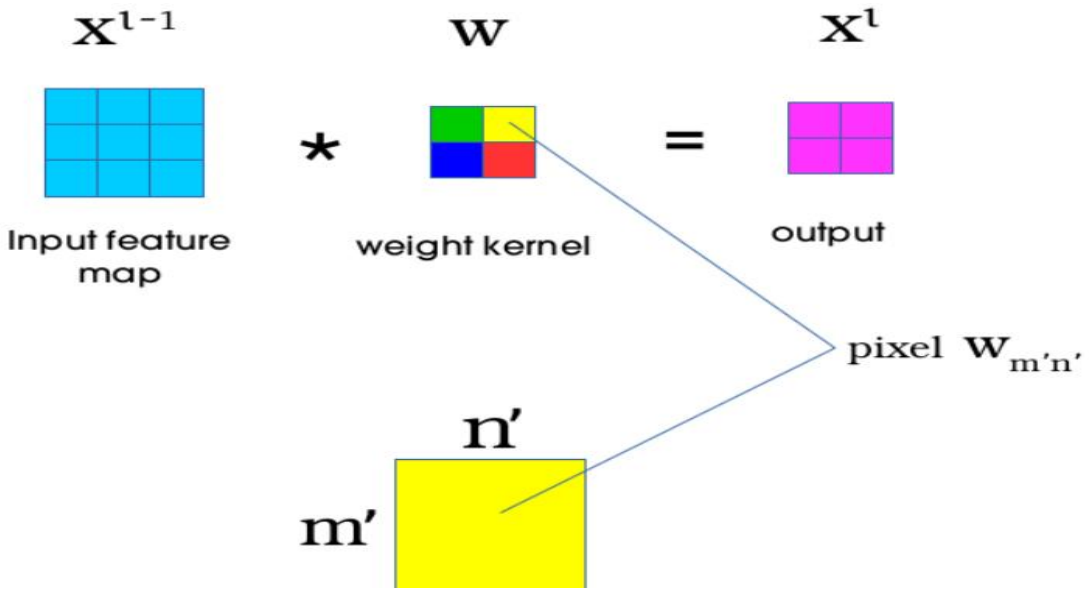
For a total of P predictions, the predicted network outputs y_p and their corresponding targeted values t_p the the mean squared error is given by:

$$E = \frac{1}{2}\sum_{p}(t_p - y_p)^2 \quad (9)$$

Backpropagation

For backpropagation there are two updates performed, for the weights and the deltas. Lets begin with the weight update.

We are looking to compute $\frac{\partial E}{\partial w_{m',n'}^{l}}$ which can be interpreted as the measurement of how the change in a single pixel $w_{m',n'}$ in the weight kernel affects the loss function E.



Convolution between the input feature map of dimension $H \times W$ and the weight kernel of dimension $k_1 \times k_2$ produces an output feature map of size $(H - k_1 + 1)$ by $(W - k_2 + 1)$. The gradient component for the individual weights can be obtained by applying the chain rule in the following way:

$$\frac{\partial E}{\partial w^l_{m',n'}} = \sum_{i=0}^{H-k_1} \sum_{j=0}^{W-k_2} \frac{\partial E}{\partial x^l_{i,j}} \frac{\partial x^l_{i,j}}{\partial w^l_{m',n'}}$$
$$= \sum_{i=0}^{H-k_1} \sum_{j=0}^{W-k_2} \delta^l_{i,j} \frac{\partial x^l_{i,j}}{\partial w^l_{m',n'}} \quad (10)$$

In Eq. 10, $x_{i,j}^l$ is equivalent to $\sum_m \sum_n w_{m,n}^l o_{i+m,j+n}^{l-1} + b^l$ and expanding this part of the equation gives us:

$$\frac{\partial x_{i,j}^l}{\partial w_{m',n'}^l} = \frac{\partial}{\partial w_{m',n'}^l}\left(\sum_m \sum_n w_{m,n}^l o_{i+m,j+n}^{l-1} + b^l\right) \tag{11}$$

Further expanding the summations in Eq. 11 and taking the partial derivatives for all the components results in zero values for all except the components where $m = m'$ and $n = n'$ in $w_{m,n}^l o_{i+m,j+n}^{l-1}$ as follows:

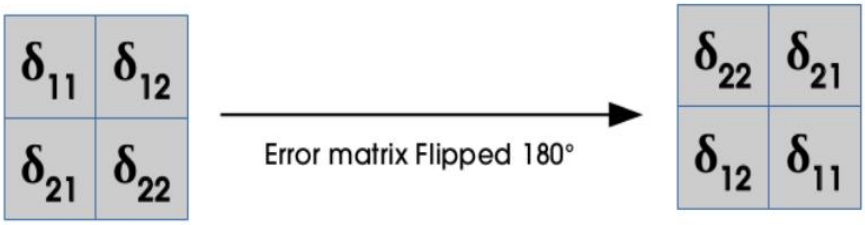
$$\begin{aligned}\frac{\partial x_{i,j}^l}{\partial w_{m',n'}^l} &= \frac{\partial}{\partial w_{m',n'}^l}\left(w_{0,0}^l o_{i+0,j+0}^{l-1} + \cdots + w_{m',n'}^l o_{i+m',j+n'}^{l-1} + \cdots + b^l\right) \\ &= \frac{\partial}{\partial w_{m',n'}^l}\left(w_{m',n'}^l o_{i+m',j+n'}^{l-1}\right) \\ &= o_{i+m',j+n'}^{l-1} \end{aligned} \tag{12}$$

Substituting Eq. 12 in Eq. 10 gives us the following results:

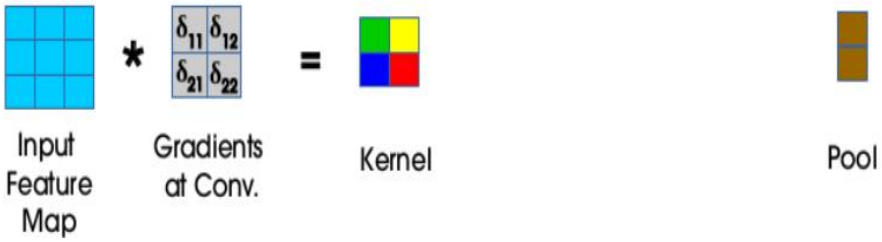
$$\frac{\partial E}{\partial w_{m',n'}^l} = \sum_{i=0}^{H-k_1} \sum_{j=0}^{W-k_2} \delta_{i,j}^l o_{i+m',j+n'}^{l-1} \tag{13}$$

$$= \mathrm{rot}_{180^\circ}\left\{\delta_{i,j}^l\right\} * o_{m',n'}^{l-1} \tag{14}$$

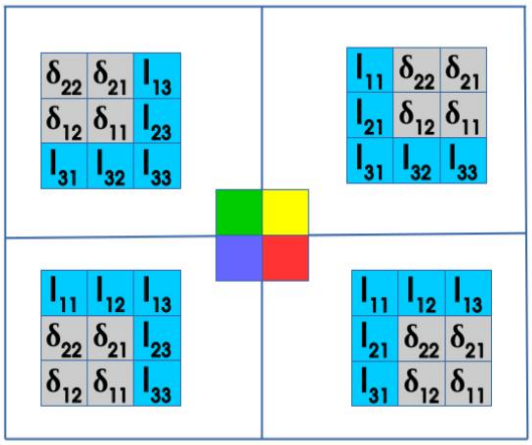
The dual summation in Eq. 13 is as a result of weight sharing in the network (same weight kernel is slid over all of the input feature map during convolution). The summations represents a collection of all the gradients $\delta_{i,j}^l$ coming from all the outputs in layer l.
Obtaining gradients w.r.t to the filter maps, we have a cross-correlation which is transformed to a convolution by "flipping" the delta matrix $\delta_{i,j}^l$ (horizontally and vertically) the same way we flipped the filters during the forward propagation.
An illustration of the flipped delta matrix is shown below:



The diagram below shows gradients $(\delta_{11}, \delta_{12}, \delta_{21}, \delta_{22})$ generated during backpropagation:



The convolution operation used to obtain the new set of weights as is shown below:



During the reconstruction process, the deltas $(\delta_{11}, \delta_{12}, \delta_{21}, \delta_{22})$, are used. These deltas are provided by an equation of the form:

$$\delta_{i,j}^{l} = \frac{\partial E}{\partial x_{i,j}^{l}} \qquad (15)$$

At this point we are looking to compute $\frac{\partial E}{\partial x_{i',j'}^{l}}$ which can be interpreted as the measurement of how the change in a single pixel $x_{i',j'}$ in the input feature map affects the loss function E.

From the diagram above, we can see that region in the output affected by pixel $x_{i',j'}$ from the input is the region in the output bounded by the dashed lines where the top left corner pixel is given by $(i'-k_1+1, j'-k_2+1)$ and the bottom right corner pixel is given by (i', j').

Using chain rule and introducing sums give us the following equation:

$$\begin{aligned}\frac{\partial E}{\partial x^l_{i',j'}} &= \sum_{i,j \in Q} \frac{\partial E}{\partial x^{l+1}_Q} \frac{\partial x^{l+1}_Q}{\partial x^l_{i',j'}} \\ &= \sum_{i,j \in Q} \delta^{l+1}_Q \frac{\partial x^{l+1}_Q}{\partial x^l_{i',j'}} \end{aligned} \tag{16}$$

Q in the summation above represents the output region bounded by dashed lines and is composed of pixels in the output that are affected by the single pixel $x_{i',j'}$ in the input feature map. A more formal way of representing Eq. 16 is:

$$\begin{aligned}\frac{\partial E}{\partial x^l_{i',j'}} &= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \frac{\partial E}{\partial x^{l+1}_{i'-m,j'-n}} \frac{\partial x^{l+1}_{i'-m,j'-n}}{\partial x^l_{i',j'}} \\ &= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \delta^{l+1}_{i'-m,j'-n} \frac{\partial x^{l+1}_{i'-m,j'-n}}{\partial x^l_{i',j'}} \end{aligned} \tag{17}$$

In the region Q, the height ranges from $i'-0$ to $i'-(k_1-1)$ and width $j'-0$ to $j'-(k_2-1)$. These two can simply be represented by $i'-m$ and $j'-n$ in the summation since the iterators m and n exists in the following similar ranges from $0 \leq m \leq k_1-1$ and $0 \leq n \leq k_2-1$.

In Eq. 17, $x^{l+1}_{i'-m,j'-n}$ is equivalent to $\sum_{m'} \sum_{n'} w^{l+1}_{m',n'} o^l_{i'-m+m',j'-n+n'} + b^{l+1}$ and expanding this part of the equation gives us:

$$\frac{\partial x^{l+1}_{i'-m,j'-n}}{\partial x^l_{i',j'}} = \frac{\partial}{\partial x^l_{i',j'}} \left(\sum_{m'} \sum_{n'} w^{l+1}_{m',n'} o^l_{i'-m+m',j'-n+n'} + b^{l+1} \right)$$

$$= \frac{\partial}{\partial x^l_{i',j'}} \left(\sum_{m'} \sum_{n'} w^{l+1}_{m',n'} f\left(x^l_{i'-m+m',j'-n+n'} \right) + b^{l+1} \right) \quad (18)$$

Further expanding the summation in Eq. 17 and taking the partial derivatives for all the components results in zero values for all except the components where $m' = m$ and $n' = n$ so that $f\left(x^l_{i'-m+m',j'-n+n'}\right)$ becomes $f\left(x^l_{i',j'}\right)$ and $w^{l+1}_{m',n'}$ becomes $w^{l+1}_{m,n}$ in the relevant part of the expanded summation as follows:

$$\begin{aligned}
\frac{\partial x^{l+1}_{i'-m,j'-n}}{\partial x^l_{i',j'}} &= \frac{\partial}{\partial x^l_{i',j'}} \left(w^{l+1}_{m',n'} f\left(x^l_{0-m+m',0-n+n'} \right) + \cdots + w^{l+1}_{m,n} f\left(x^l_{i',j'} \right) - \right. \\
&= \frac{\partial}{\partial x^l_{i',j'}} \left(w^{l+1}_{m,n} f\left(x^l_{i',j'} \right) \right) \\
&= w^{l+1}_{m,n} \frac{\partial}{\partial x^l_{i',j'}} \left(f\left(x^l_{i',j'} \right) \right) \\
&= w^{l+1}_{m,n} f'\left(x^l_{i',j'} \right)
\end{aligned}$$

Substituting Eq. 19 in Eq. 17 gives us the following results:

$$\frac{\partial E}{\partial x^l_{i',j'}} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \delta^{l+1}_{i'-m,j'-n} w^{l+1}_{m,n} f'\left(x^l_{i',j'} \right) \quad (20)$$

For backpropagation, we make use of the flipped kernel and as a result we will now have a convolution that is expressed as a cross-correlation with a flipped kernel:

$$\begin{aligned}
\frac{\partial E}{\partial x^l_{i',j'}} &= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \delta^{l+1}_{i'-m,j'-n} w^{l+1}_{m,n} f'\left(x^l_{i',j'} \right) \\
&= \mathrm{rot}_{180^\circ} \left\{ \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} \delta^{l+1}_{i'+m,j'+n} w^{l+1}_{m,n} \right\} f'\left(x^l_{i',j'} \right) \quad (21) \\
&= \delta^{l+1}_{i',j'} * \mathrm{rot}_{180^\circ} \left\{ w^{l+1}_{m,n} \right\} f'\left(x^l_{i',j'} \right) \quad (22)
\end{aligned}$$

Pooling Layer

The function of the pooling layer is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting. No learning takes place on the pooling layers.
Pooling units are obtained using functions like max-pooling, average pooling and even L2-norm pooling. At the pooling layer, forward propagation results in an N×N pooling block being reduced to a single value - value of the "winning unit". Backpropagation of the pooling layer then computes the error which is acquired by this single value "winning unit".

To keep track of the "winning unit" its index noted during the forward pass and used for gradient routing during backpropagation. Gradient routing is done in the following ways:

- **Max-pooling** - the error is just assigned to where it comes from - the "winning unit" because other units in the previous layer's pooling blocks did not contribute to it hence all the other assigned values of zero
- **Average pooling** - the error is multiplied by $\frac{1}{N \times N}$ and assigned to the whole pooling block (all units get this same value).

References

1. https://www.geeksforgeeks.org/backpropagation-in-convolutional-neural-networks/
2. Deep Learning Step by Step with Python, N D Lewis, 2016